Assignment 2 Report
DD2438

GROUP 7

Gawtam C R Zyad Haddad

12-10-2001 04-04-2002

gawtam@kth.se zyad@kth.se

Abstract

Multi-agent pathfinding problems, where a set of autonomous agents such as cars or drones must navigate a shared space to reach their respective goal positions without colliding, are central to a wide range of real-world systems, from automated warehouses to aerial drone fleets. The core objective is to minimize the overall makespan—the time until all agents reach their goals—while guaranteeing safety and coordination. To address this challenge, we implemented the Conflict-Based Search (CBS) algorithm, a state-of-the-art approach that decouples planning and resolves conflicts via a hierarchical constraint tree. CBS incrementally identifies and resolves pairwise collisions, enabling scalable and optimal coordination in complex environments. Our implementation was evaluated on a suite of benchmark scenarios (various terrains incorporating different constraints), revealing both the algorithm's strengths and the limitations of our current system. While our model exhibits clear shortcomings in performance and robustness under certain conditions, the experiments reaffirm CBS's potential as a practical and theoretically grounded solution. These findings underscore the importance of principled conflict resolution in multi-agent planning and its implications for next-generation autonomous systems.

1 Introduction

Multi-Agent Pathfinding (MAPF) addresses the challenge of coordinating autonomous agents—such as drones or robots—in shared environments to reach designated goals without collisions. This problem is foundational to applications like warehouse automation, drone delivery, and autonomous vehicle fleets, where minimizing the *makespan* (time for all agents to complete tasks) is critical. MAPF is NP-hard [7], and its complexity scales combinatorially with the number of agents, necessitating algorithms that balance computational efficiency with collision-free guarantees.

Traditional solutions fall into two paradigms. Centralized planners, such as Conflict-Based Search (CBS) [6], resolve conflicts in a global state space, ensuring optimality at the cost of scalability. Decentralized methods, like Reciprocal Velocity Obstacles (RVO) [3], prioritize real-time reactivity but risk deadlocks in dense scenarios. CBS bridges this divide by hierarchically decoupling planning: low-level A^* generates individual paths, while a high-level constraint tree iteratively resolves conflicts. Though CBS guarantees completeness and optimality, its performance degrades in dynamic or kinematically constrained environments. This work evaluates CBS in a Unity3D simulation with holonomic (drone) and non-holonomic (car) agents and introduces a novel "bakery" scenario where teams of agents dynamically share goals, mirroring real-world delivery fleets.

1.1 Problem Statement

The project tackles two MAPF variants. In the *standard problem*, N agents navigate a 2D grid from unique start positions to fixed goals. Collisions occur if agents occupy the same vertex (vertex conflict) or traverse opposing edges (edge conflict) simultaneously. The objective is to minimize makespan while ensuring collision-free paths.

The *bakery problem* extends this framework: agents are grouped into teams, each assigned a shared pool of customer goals. Any agent in a team can serve any customer, but goals are cleared once visited. This introduces dynamic goal assignment, where agents must be allocated tasks to minimize total completion time. Key challenges include scalability (resolving conflicts for $N > 20$ agents), kinodynamic constraints (non-holonomic car paths), and dynamic goal reallocation.

1.2 Contributions

This work makes three contributions. First, we implement and analyze Conflict-Based Search (CBS) with constraint prioritization heuristics, providing pedagogical insights into its practical limitations. Second, we extend CBS to bakery scenarios through a greedy goal assignment strategy that minimizes accumulated travel costs. Third, we empirically demonstrate how kinematic constraints and congestion degrade CBS performance, motivating hybrid planning approaches.

1.3 Report Structure

Section 2 reviews centralized and decentralized MAPF methods. Section 3 details our CBS implementation and bakery goal allocation strategy. Sections 4–5 present experimental results and failure analysis. Section 6 concludes with recommendations for future work.

2 Related Work

The Multi-Agent Pathfinding (MAPF) problem has been extensively studied due to its relevance in robotics, logistics, and autonomous systems. Existing approaches broadly fall into three categories: **centralized**, **decentralized**, and **hybrid** methods, each with distinct trade-offs in optimality, scalability, and applicability to dynamic environments.

Centralized methods, such as Conflict-Based Search (CBS) [6], treat MAPF as a joint planning problem in a high-dimensional state space. CBS employs a two-level hierarchy: a low-level A^* planner generates individual agent paths, while a high-level constraint tree resolves conflicts iteratively. This guarantees completeness and optimality but incurs exponential time complexity in worst-case scenarios. Subsequent variants like Enhanced CBS (ECBS) [1] and Prioritized CBS [2] mitigate computational overhead by introducing suboptimality bounds or prioritizing critical conflicts. While centralized methods excel in static environments with full observability—as in our simulation—they struggle with scalability in highly dynamic or partially observable settings [3].

Decentralized methods, such as Reciprocal Velocity Obstacles (RVO) and Optimal Reciprocal Collision Avoidance (ORCA) [3], enable agents to compute collision-free velocities locally using reactive rules. These approaches scale efficiently to large agent populations and handle real-time dynamics but lack global coordination, often resulting in suboptimal paths or deadlocks

in congested environments. For instance, ORCA's reliance on velocity adjustments may fail in narrow corridors or symmetric scenarios where agents oscillate indefinitely [3]. Such limitations make decentralized methods less suitable for structured, high-density simulations like ours, where global coordination is critical.

Hybrid approaches aim to combine the strengths of centralized and decentralized paradigms. Techniques like Windowed CBS [4] replan paths periodically within finite time horizons, while others integrate local collision avoidance with global planners to handle dynamic obstacles [5]. Recent work by Li et al. [4] extends CBS to lifelong MAPF scenarios, dynamically assigning tasks to agents while resolving conflicts. Though promising, hybrid methods often require complex synchronization between planners and controllers, complicating implementation in resource-constrained systems.

Beyond algorithmic distinctions, MAPF research has increasingly focused on **real-world constraints**. Kinodynamic extensions of Hybrid A^* [5] generate dynamically feasible trajectories for non-holonomic vehicles, while temporal CBS variants [2] account for acceleration and braking limits. However, in simulation environments with idealized agent dynamics—such as our discrete grid-based setup—classical CBS remains preferable due to its simplicity and theoretical guarantees [6].

Our work builds on these foundations by implementing CBS in a Unity3D simulation with teams of holonomic (drones) and non-holonomic (cars) agents. While prior studies like Cohen et al. [2] explore constraint prioritization, we empirically validate CBS's sensitivity to agent density and map topology, particularly in the novel "bakery" scenario where goals are assigned to teams. This extends traditional MAPF benchmarks, which typically assume fixed agent-goal pairs [6].

3 Proposed Method

This section details the implementation of Conflict-Based Search (CBS) for solving the multi-agent pathfinding (MAPF) problem, with a focus on our system architecture, constraint handling strategy, and resolution of deadlock-like situations.

3.1 Conflict-Based Search Overview

CBS is a two-level algorithm widely used in MAPF for generating optimal solutions with bounded computational complexity. At the low level, each agent independently computes a shortest path using A* search. At the high

level, CBS manages a constraint tree (CT), in which each node stores a set of space-time constraints and corresponding agent paths that satisfy them.

If no conflicts are detected among the agent paths, the solution is returned. Otherwise, CBS identifies the earliest conflict and branches the tree, introducing new constraints to prevent the conflict in each child node. This process is repeated recursively until a conflict-free solution is found. The algorithm is complete and optimal, and its layered design makes it more scalable than fully coupled approaches [4].

3.2 Implementation Details

Our implementation closely follows the canonical CBS framework while incorporating modern best practices inspired by recent advancements. Constraint tree nodes are prioritized by their makespan (the maximum path length among all agents), and a min-heap is used to manage node expansion efficiently. Each node consists of:

- A cumulative list of constraints;
- A set of individually computed agent paths;
- A makespan-based cost value used for prioritization.

We used a variant of time-aware A* that respects constraints dynamically, as described by Ma et al. [5]. This allows flexible re-planning of individual agents as constraints evolve. To further reduce redundant computation, we reuse prior valid paths and only re-plan agents affected by newly introduced constraints.

Our map environment is a discrete 2D grid with uniform cost cardinal movements. Agents may wait in place, and time advances in synchronous ticks. The system assumes that all agents move simultaneously, as is standard in MAPF literature.

3.3 Low-Level Path Planning and Following

To generate dynamically feasible motion for individual agents (drones or cars), we used a Hybrid A* planner combined with a waypoint-following mechanism. Hybrid A* extends classical A* to continuous state spaces by incorporating orientation and vehicle kinematics into the search process, enabling non-holonomic agents to follow realistic trajectories. This method is particularly suited to autonomous driving and drone applications, where

Algorithm 1 Conflict-Based Search (CBS)

Input: Set of agents $A = \{a_1, \ldots, a_n\}$ with start and goal positions, grid map G
Output: Set of conflict-free paths $\Pi = \{\pi_1, \ldots, \pi_n\}$
Initialize constraint tree $\mathcal{CT}$
N_{root}.constraints $\leftarrow \emptyset$
for each agent $a_i \in A$ **do**
    $\pi_i \leftarrow$ A* search from start to goal for a_i with no constraints
    $N_{\text{root}}.\pi_i \leftarrow \pi_i$
end for
N_{root}.cost $\leftarrow$ Makespan(Π)
Insert N_{root} into priority queue $\mathcal{OPEN}$ ordered by cost
while $\mathcal{OPEN}$ is not empty **do**
    $N \leftarrow$ Pop node from $\mathcal{OPEN}$ with lowest cost
    $(a_i, a_j, v, t) \leftarrow$ DetectConflict($N.\Pi$)
    if no conflict **then**
        return $N.\Pi$
    end if
    for each agent $a \in \{a_i, a_j\}$ **do**
        $N' \leftarrow$ Deep copy of N
        Add constraint (a, v, t) to N'.constraints
        $\pi'_a \leftarrow$ A* search for a considering N'.constraints
        if π'_a is not failure **then**
            Replace $N'.\pi_a \leftarrow \pi'_a$
            Update N'.cost $\leftarrow$ Makespan($N'.\Pi$)
            Insert N' into $\mathcal{OPEN}$
        end if
    end for
end while
return failure (no solution found)

Figure 1: Pseudocode for the Conflict-Based Search (CBS) algorithm. The high-level search builds a constraint tree (CT) by resolving agent conflicts incrementally. Each child node adds a new constraint and triggers partial replanning.

simple grid-based paths are insufficient. Rather than implementing this component from scratch, we integrated the Hybrid A* and waypoint controller from Group 6's code for Assignment 1, which provided a tested foundation.

3.4 Constraint Management

Constraints are represented as tuples of the form `(agent, location, timestep)` for vertex conflicts, and `(agent, from, to, timestep)` for edge conflicts. When two agents attempt to occupy the same location at the same timestep, or cross edges simultaneously in opposite directions, we generate two new constraint tree nodes, each forbidding one agent from executing the conflicting move.

This dual-branching mechanism allows the high-level search to systematically explore conflict-resolving alternatives. Our implementation follows the constraint resolution framework introduced by Felner et al. [3], which emphasizes prioritizing ca conflicts to improve convergence and pruning of redundant branches.

We also utilize a conflict detection heuristic known as the "earliest conflict first" policy to minimize branching depth and accelerate convergence [2].

3.5 Deadlock and Path Locking Mitigation

Although CBS guarantees completeness, practical implementations can suffer from lock-in scenarios where repeated constraint addition prevents feasible path generation—especially in narrow corridors or high-density regions.

To address this, we implemented several mitigation strategies:

- **Goal Padding:** Agents that arrive early remain idle at goal positions but can be forced to move if they create conflicts. We mitigate this by padding goals with wait actions while ensuring flexibility in case conflict resolution requires slight repositioning.
- **Node Pruning via Conflict Prioritization:** Inspired by techniques from Cohen et al. [2], we avoid generating branches on non-cardinal or semi-cardinal conflicts in early CT levels to reduce unnecessary divergence.
- **Fail-Safe Bound:** A maximum node expansion count was set to avoid infinite loops in pathological cases. When this bound is reached, we return the best available node with minimal unresolved conflicts.

3.6 Bakery Teams Solution

As we focused most of our time on CBS, we chose to implement a simple solution given the time constraints. The problem of assigning multiple team goals to autonomous agents can be addressed using a greedy heuristic that

minimizes the overall travel time until all agents have passed their respective goal positions. In this approach, each team is considered as a collection of agents, such as cars or drones, and a set of customer goals. Each agent a_i has an initial start position s_i and may already have one or more assigned goals. The effective starting position for an agent is defined as its original start position if no goals have been assigned, or the position of its last assigned goal g_i^{last} otherwise. The Euclidean distance between an agent's effective position and a goal g_j is computed as

$$d_{ij} = \|p_i - g_j\|,$$

where p_i represents the effective starting position of agent a_i.

Each agent also maintains an accumulated travel distance T_i, and the total cost of reaching a goal g_j is expressed as

$$D_{ij} = T_i + d_{ij}.$$

For each goal, the algorithm iterates over all agents within the same team and assigns the goal to the agent that minimizes D_{ij}. Once a goal is assigned, the corresponding agent's accumulated travel distance is updated according to the distance to the newly assigned goal, so that

$$T_i \leftarrow T_i + d_{ij}.$$

This simple, iterative procedure ensures that goals are allocated to the closest available agent in terms of both current position and previously traveled distance.

In addition to the initial assignment, the system also incorporates a mechanism to handle path failures. If an agent fails to compute a valid path to a goal, the problematic goal is removed from the agent's list and reassigned to the next best candidate. The candidate selection is again based on the minimum value of D_{ij}, ensuring that the additional travel cost is minimized when reassigning the goal.

Although the greedy method does not guarantee a globally optimal solution, it offers a computationally efficient way to distribute the workload among agents. By continually updating the accumulated travel cost T_i after each assignment, the algorithm naturally balances the routes among team members. This makes the approach practical for real-time applications in scenarios where multiple autonomous vehicles share a common workspace, as it reduces the overall time required for all agents to complete their tasks while keeping the computational complexity low.

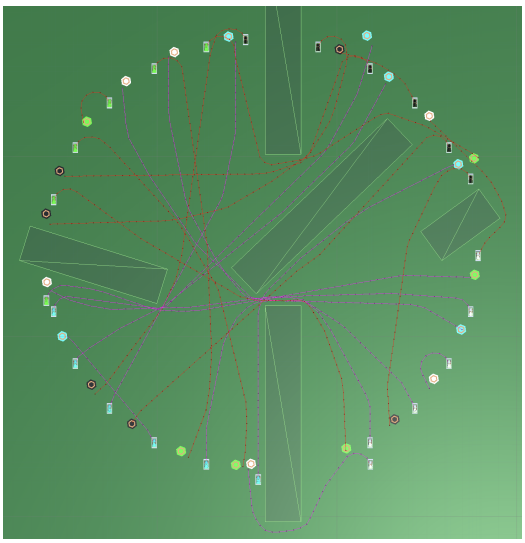

Figure 2: Paths generated for open bakery map

4 Experimental Results

This section presents an analysis of CBS-based planning against other approaches developed during the course. The benchmarks are divided into two categories: **Car Model** (requiring non-holonomic path planning and realistic motion constraints) and **Drone Model** (fully holonomic agents with freedom in direction). Each group's solution was tested on 14 scenarios, 7 under each model.

4.1 Experimental Setup

All solutions were tested under equivalent simulation environments using identical start-goal configurations and map structures. Group 7 (our team) employed Conflict-Based Search (CBS) as the global planning method, extended with collision-aware trajectory registration.

4.2 Analysis of Outcome

Group 7 (our implementation) adopted a fully global planning approach using Conflict-Based Search (CBS) for coordination and Hybrid A* for motion generation. This method showed promising results in simpler scenarios, but failed to complete the majority of the high-complexity benchmarks within the allowed time, particularly under the Car Model. This outcome highlights

[1]Group 7's results reflect a sequential dispatch system.

Table 1: Benchmark results for Car Model (2000 indicates failure)

Group	open	open_blocks	intersection	highway	onramp	bakeries	city_bakery
G1	29.32	28.34	65.30	45.00	45.88	31.42	87.62
G4	66.80	100.22	155.08	109.70	88.80	56.31	291.30
G5	133.97	219.79	214.47	2000.00	2000.00	2000.00	2000.00
G7 **??**	145.79	230.23	412.38	2000.00	2000.00	2000.00	2000.00
G8	69.28	36.94	94.80	100.54	104.02	39.24	121.72
G17	2000.00	444.32	807.60	2000.00	2000.00	2000.00	2000.00
G18	47.00	63.00	268.00	177.00	249.00	2000.00	2000.00

Table 2: Benchmark results for Drone Model (2000 indicates failure)

Group	open	open_blocks	intersection	highway	onramp	bakeries	city_bakery
G1	20.18	19.26	47.38	46.04	46.80	21.54	42.00
G4	66.90	100.20	155.10	105.10	88.90	56.31	223.00
G5	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00
G7 [1]	187.90	165.89	312.67	345.87	305.23	2000.00	2000.00
G8	52.52	26.76	72.80	79.72	80.36	29.24	94.42
G17	268.30	182.38	468.86	555.10	376.90	2000.00	2000.00
G18	80.00	84.00	175.00	204.00	222.00	2000.00	2000.00

both the strengths and limitations of pure global planning when not combined with local fallback mechanisms or dynamic replanning.

One key characteristic of our approach was its reliance on a centralized trajectory planner that registered paths in a time-space dictionary. Prior to committing to a trajectory point, the planner checked for conflicts with other agents' future paths. If a conflict was detected, the agent stalled for several time steps before retrying, introducing a form of passive avoidance. While this worked well in sparse environments, it led to cascading delays or deadlocks in more congested cases.

In contrast, Group 1 consistently outperformed others due to their efficient global planner and heuristic-guided conflict avoidance. Their use of a "crowded point" penalty function helped disincentivize bottlenecks, a strategy we could benefit from. Additionally, their ability to resolve all scenarios without local collision avoidance demonstrated the effectiveness of refined heuristics and prioritized constraint resolution in CBS.

Group 6's approach — performed reasonably well but suffered from suboptimal coordination. Their lack of a central planner led to better performance than ours in some drone scenarios but made their system vulnerable in dense car traffic.

Group 8's system, which combined Hybrid A* with topological sorting and Simulated Annealing for MDVRP, had significantly better success rates than ours. Their use of goal assignment optimization and delayed launch

scheduling allowed for better handling of start congestion and resource balancing — features absent in our current system.

5 Analysis of Failure

Even though the final system was built on a strong theoretical foundation—namely Conflict-Based Search (CBS) and Hybrid A*—it fell short in real-world execution, particularly in the car-based scenarios. This section dissects the technical failures that led to this outcome, highlights when these issues became apparent, and reflects on what could have been done differently. We also articulate the key lessons we've drawn from the experience to avoid similar pitfalls in future projects.

5.1 Technical Aspect of Failure

Throughout the project, our group focused heavily on implementing a robust Conflict-Based Search (CBS) planner, which became the cornerstone of our solution. While this was essential for tackling the multi-agent coordination problem, it introduced a significant imbalance in our planning-progress-risk triad.

In the early stages, CBS implementation consumed most of our attention, and while initial results were promising in static grid scenarios, we underestimated the complexity introduced by the car dynamics and path tracking. The technical risk became evident during integration testing, particularly in complex Car Model scenarios, where agents failed to follow paths reliably due to accumulated tracking error or inefficient stop-go behavior. At this point, it became clear that failure was not just a theoretical risk but a likely outcome if corrective measures were not taken quickly.

A major design flaw emerged in our motion execution: we treated stop-go sequences like "Driving–Stop–Driving" identically to "Stop–Driving–Driving", leading to inefficient path execution. Most agents began by waiting at their start position, which initially reduced early collisions but ultimately led to heavy congestion and timing conflicts downstream. This naive approach to motion timing severely impacted our performance in both congested and sparse environments.

Another overlooked technical component was the trajectory tracking system. We relied heavily on the PD tracker provided by earlier groups (notably Group 6), only tuning the proportional and derivative gains (k_p, k_d) without fundamentally questioning its adequacy for our dynamics. We missed an opportunity to explore more advanced or adaptive tracking strategies, such

as ABS-like braking logic to improve deceleration smoothness and stop precision, adaptive control to dynamically adjusting gains based on curvature or speed and preview control planning based on future waypoints, not just immediate heading.

These omissions meant that while our global planner produced viable paths, our agents failed to track them accurately, especially at intersections and turn-heavy environments. The performance gap between Drone and Car models in our results directly reflects this mismatch: our drone trajectories, which did not require tight tracking, succeeded more often, while the car dynamics exposed our system's weaknesses.

5.2 Management Aspect of Failure

From a project management perspective, our team fell into a common trap: overcommitting to one subsystem (CBS) while underestimating the time required to integrate and validate others. While we had an ambitious vision early on, we didn't properly reassess scope once we encountered difficulties in CBS edge cases. As a result, our energy and time went disproportionately into tuning the planner—even as critical components like tracking and behavior logic remained underdeveloped.

We also missed key project management checkpoints. For example:

- We did not define clear milestones for when integration testing should begin.
- Risk analysis was not done systematically.
- No explicit fallback plan was established in case CBS performance became a bottleneck.

Additionally, our team placed high trust in legacy components (e.g., the PD tracker) without questioning their compatibility. We spent too little time brainstorming alternatives such as adaptive tracking, model-predictive control, or even simpler rule-based trajectory smoothing.

This imbalance resulted in a last-minute rush to patch issues in motion control and led to missed opportunities for performance improvement in high-stakes benchmarks.

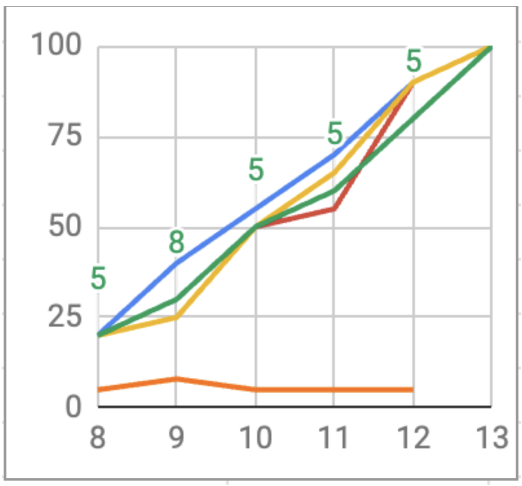

Figure 3: "Progress-report" plot with planned input in time, actual input in time and progress and risk estimate.

5.3 Lessons Learned

While technical missteps were the primary cause of our system's underperformance, the deeper issue was one of project management and decision-making under time constraints. In hindsight, our planning process lacked checkpoints to evaluate risk across subsystems and adjust our time allocation accordingly.

This project highlighted a key lesson in system-level thinking: success is not determined by the quality of one subsystem, but by the integration and balance of all parts. Our strong focus on CBS planning blinded us to downstream bottlenecks in tracking and execution. In future projects, we will:

- Allocate dedicated time early for system architecture reviews that explicitly analyze risks in each subsystem.
- Prototype all subsystems in parallel at low fidelity before committing to final implementations.
- Run "integration sprints" earlier in the timeline to surface timing and control issues sooner.
- Avoid blind reuse of legacy components (like the PD tracker) without understanding their limitations and trade-offs.

By applying these lessons, we aim to develop more balanced, resilient, and agile approaches to multi-agent systems and real-time robotics in the future.

6 Summary and Conclusions

Key insights for improvement include:

- **Decentralized Fallback**: Incorporating a reactive collision avoidance layer or decentralized replanning would have improved success rates in scenarios where global plans failed due to unforeseen interactions.
- **Heuristic Prioritization**: Our planner lacked prioritization of critical agents. Adopting strategies such as "longest path first" (seen in Group 6) or conflict cardinality-based branching could significantly reduce constraint tree size.
- **Delayed Launches**: Implementing delayed starts based on conflict heatmaps or start-zone congestion would reduce early-time conflicts and minimize stalling.

In summary, while our CBS-based framework was structurally sound, the absence of dynamic adaptations — such as fallback mechanisms or adaptive launch strategies — limited its effectiveness in real-world complexity scenarios. These observations point toward hybrid approaches that combine global coordination with local robustness as a promising direction for future work.

References

[1] Max Barer, Guni Sharon, Roni Stern, and Ariel Felner. Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem. In Eighth Annual Symposium on Combinatorial Search (SoCS), 2014.

[2] Liron Cohen, Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Prioritized planning for multi-agent pathfinding with conflict-based search. Journal of Artificial Intelligence Research, 73:727–778, 2022.

[3] Ariel Felner, Roni Stern, Solomon E Shimony, Eldar Boyarski, Guni Sharon, T. K. Satish Kumar, and Sven Koenig. Search-based optimal solvers for the multi-agent pathfinding problem: Summary and challenges. Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS), pages 1179–1183, 2018.

[4] Jiaoyang Li, Hang Ma, T. K. Satish Kumar, and Sven Koenig. Generalized and improved conflict-based search. In Proceedings of the

International Joint Conference on Artificial Intelligence (IJCAI), pages 1201–1209, 2021.

[5] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. Proceedings of the AAAI Conference on Artificial Intelligence, 33(01):8485–8492, 2019.

[6] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R Sturtevant. Conflict-based search for optimal multi-agent pathfinding. Artificial Intelligence, 2015.

[7] Jingjin Yu and Steven M LaValle. Planning optimal paths for multiple robots on graphs. 2013 IEEE International Conference on Robotics and Automation, 2013.